



UNIVERSIDAD INTERNACIONAL DEL ECUDOR

Ingeniería en Sistemas

ASIGNATURA

**FUNDAMENTOS DE LA SEGURIDAD DE LOS SISTEMAS
INFORMATICOS Y DE LA INFORMACIO**

TEMA

APIs y Base de Datos

FECHA DE ELABORACIÓN
13-05-2025

ELABORADO POR

Mateo Steve Cantos Muñoz

INDICE

Objetivo:

Implementar un sistema de Base de datos que contenga identidades y autenticación desarrollada con FasAPI, SQL models y aplicando técnicas de encriptación y criptografía moderna tipo (Hashing, Salting y Peppering a través de bcrypt).

Etapa 1

1. Por que no usar MD5 o SHA-256?

Los algoritmos de Hash criptográficos como el MD y el SHA, fueron hechos principalmente para procesar volúmenes grandes de datos de forma rápida.

Característica	MD5 / SHA-256	bcrypt / Argon2
Velocidad	Muy rápido (GH/s)	Lento (diseñado así)
Sal (Salt)	No integrada	Integrada automática
Costo ajustable	No	Sí (factor de costo)
Resistencia GPU/ASIC	Muy baja	Alta
Uso recomendado	Integridad de datos	Contraseñas

Por que no se deben usar MD5 o SHA-256?

Una de las razones principales es que MD5 en el ámbito de la seguridad es que fue diseñado mas que nada para verificar la integridad de los datos mas, es decir comprobar si un archivo grande se descargó correctamente. Esto lo consiguieron ya que fueron programados y optimizados matemáticamente para ser extremadamente rápidos y consumir la menor cantidad de recursos de CPU posible. Un atacante puede probar diccionarios completos o todas las combinaciones posibles en cuestión de horas o días.

MD5 y SHA-256 ejecutan su cálculo en una sola pasada. En cambio obliga al procesador a realizar los cálculos matemáticos miles de veces antes de devolver el Hash final.

En qué momento se añade el Pepper y por qué no se guarda en la base de datos?

Pepper es una capa mas de seguridad adicional que es directamente conectada a la contraseña justo antes de procesarla con el algoritmo del hash. El pepper se mantiene fuera de la base de datos, típicamente como una variable de entorno del servidor. Esto se logra gracias a la separación de capas, es decir que si un

atacante logra acceder y descargar completamente la base de datos (ataque SQL injection, fuga de backup, etc.), sin el Pepper no puede intentar ataques de diccionario o fuerza bruta efectivos, porque cada intento requiere conocer el Pepper para generar el hash correcto. El Pepper permanece protegido en el servidor de aplicaciones, no en el de base de datos. Los pasos de como funciona:

- El usuario proporciona su contraseña en texto plano.
- El sistema obtiene el Pepper desde la variable de entorno (ej: SECRET_PEPPER) y lo concatena a la contraseña: contraseña + PEPPER.
- Se aplica bcrypt o Argon2 al resultado. Estos algoritmos generan internamente un Salt aleatorio único para cada operación.
- Solo se guarda en la base de datos el hash resultante (que incluye el Salt internamente). El Pepper NUNCA se almacena en la base de datos.
- Al autenticar, se repite el proceso: se concatena el Pepper a la contraseña ingresada y se verifica contra el hash almacenado.

Etapa 2

1. Evidencias y desarrollo del proyecto.

```
class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str = Field(unique=True, index=True)
    hashed_password: str

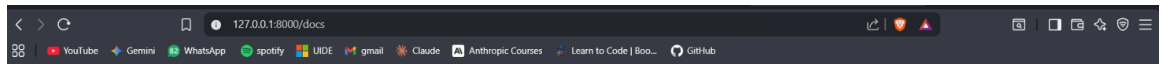
class CrearUsuario(BaseModel):
    username: str
    password: str
```

Configuran los modelos de SQLAlchemy, incluyendo la definición de las tablas de usuario

```
def contrahash(password: str) -> str:
    peppered_password = password + PEPPER
    return pwd_context.hash(peppered_password)

def verificarcontra(plain_password: str, hashed_password: str) -> bool:
    peppered_password = plain_password + PEPPER
    return pwd_context.verify(peppered_password, hashed_password)
```

Esta es la parte donde interviene el Hashing con la unión del Pepper, donde se obtiene las variables del entorno en ese caso en el .env donde definimos cual va a ser la Key.



Api 0.1.0 OAS 3.1
/openapi.json

default

POST /register Register User

POST /login Login User

Schemas

HTTPValidationError > Expand all object

UserCreate > Expand all object

ValidationError > Expand all object



Parte de la creación de la APIs, y funcionando para posteriormente la creación del Usuario y contraseña.

POST /register Register User

Parameters

No parameters

Request body ^{required} application/json

Edit Value | Schema

```
{  "username": "Samita",  "password": "Pamacita"}
```

Execute Clear

Responses

No parameters

Request body ^{required} application/json

Edit Value | Schema

```
{  "username": "Patep",  "password": "Pamacita"}
```

Execute Clear

The screenshot displays the DBeaver 26.0.3 application window. The main area shows a table named 'user' from the 'database.db' database. The table has three columns: 'id' (integer), 'username' (text), and 'hashed_password' (text). Two rows of data are visible: one with 'id' 1 and 'username' 'Matep', and another with 'id' 2 and 'username' 'Samita'. The 'hashed_password' column contains long, complex strings. The interface includes a sidebar on the left with a tree view of the database structure, a top menu bar, and a bottom status bar. The status bar indicates that 2 rows were fetched on 2026-05-14 at 13:16:15.

id	username	hashed_password
1	Matep	\$2b\$12\$kcLeM71p25g2IA03RBis/O1y6Z5nj
2	Samita	\$2b\$12\$NkmdPvfCxAmb02NhMBYCxOsbw

[illegible]

Conclusiones:

El poder usar algoritmos como MD5 o SHA para el almacenamiento de contraseña es algo inseguro ya que por la velocidad que tiene facilita los ataques de fuerza bruta y de diccionarios básicos.

El Pepper añade una capa adicional de seguridad al mantener un secreto fuera de la base de datos, aplicando el principio de defensa en profundidad. Sin acceso a las variables y si es hackeada, la base de datos solo hace visible la mínima en ese caso el user. Aunque para poder agregar mas seguridad podemos hashear incluso el user, en caso de ser por ejemplo el correo electrónico si la base de datos es hackeada no tendrían información ya que esta todo hasheado.

Los algoritmos bcrypt incorporan de serie Salt y un factor de costo que los hace bastante resistente a los ataques modernos con hardware.

En base a la auditoria, como pudimos observar, la base de datos SQLite demostró el valor crítico del salting. Ya que al evidenciar que tenemos dos mismas contraseñas pero distintos hashing, y pudimos comprobar que la asignación de una cadena aleatoria única para cada registro neutraliza por completo la amenaza

Referencias:

Larque, J. (2024, 4 junio). *Tipos de bases de datos: cuáles hay y por qué es importante elegirlos bien*. Blog de Hiberus. <https://www.hiberus.com/crecemos-contigo/tipos-de-bases-de-datos-cuales-hay-y-por-que-es-importante-elegirlos-bien/>

Arsys. (2025, 5 junio). *Tipos de bases de datos que existen*.

<https://www.arsys.es/blog/tipos-de-bases-de-datos-que-existen>

Navarrete, R. (2025, 17 diciembre). ¿Qué es el hash en ciberseguridad?

Consultoría Informática. <https://consultoriainformatica.net/que-es-el-hash-en-ciberseguridad/>

Kumar, A. (2025, 11 noviembre). *What is SHA? What is SHA used for?* |

Encryption Consulting. Encryption Consulting LLC.

<https://www.encryptionconsulting.com/education-center/what-is-sha/>

Yiga, N. (2024, 20 junio). *¿Qué es bcrypt?* Skysnag.

<https://www.skysnag.com/es/blog/what-is-bcrypt/>